WEEK-1

1. Simulate the following CPU scheduling algorithms a) FCFS b) SJF

FCFS CPU SCHEDULING ALGORITHM

```
#include<stdio.h>
#include<conio.h>
main()
{
int bt[20], wt[20], tat[20], i, n;
float wtavg, tatavg;
clrscr();
printf("\nEnter the number of processes -- ");
scanf("%d", &n);
for(i=0;i<n;i++)
{
printf("\nEnter Burst Time for Process %d -- ", i);
scanf("%d", &bt[i]);
}
wt[0] = wtavg = 0;
tat[0] = tatavg = bt[0];
for(i=1;i<n;i++)
{
wt[i] = wt[i-1] +bt[i-1];
tat[i] = tat[i-1] +bt[i];
wtavg = wtavg + wt[i];
tatavg = tatavg + tat[i];
}
printf("\t PROCESS \tBURST TIME \t WAITING TIME\t TURNAROUND TIME\n");
for(i=0;i<n;i++)
printf("\n\t P%d \t\t %d \t\t %d \t\t %d", i, bt[i], wt[i], tat[i]);
printf("\nAverage Waiting Time -- %f", wtavg/n);
printf("\nAverage Turnaround Time -- %f", tatavg/n);
getch();
}
```

INPUT

```
Enter the number of processes -- 3
Enter Burst Time for Process 0 -- 24
Enter Burst Time for Process 1 -- 3
Enter Burst Time for Process 2 -- 3
```

OUTPUT

PROCESS BURST TIME WAITING TIME TURNAROUND TIME
P0 24 0 24
P1 3 24 27
P2 3 27 30
Average Waiting Time-- 17.000000
Average Turnaround Time -- 27.000000

SJF CPU SCHEDULING ALGORITHM

```
#include<stdio.h>
#include<conio.h>
main()
{
int p[20], bt[20], wt[20], tat[20], i, k, n, temp;
float wtavg, tatavg;
clrscr();
printf("\nEnter the number of processes -- ");
scanf("%d", &n);
for(i=0;i<n;i++)
{
p[i]=i;
printf("Enter Burst Time for Process %d -- ", i);
scanf("%d", &bt[i]);
}
for(i=0;i<n;i++)
for(k=i+1;k<n;k++)
if(bt[i]>bt[k])
{
temp=bt[i];
bt[i]=bt[k];
bt[k]=temp;
}
wt[0] = wtavg = 0;
tat[0] = tatavg = bt[0];
for(i=1;i<n;i++)
{
temp=p[i];
p[i]=p[k];
p[k]=temp;
wt[i] = wt[i-1] +bt[i-1];
tat[i] = tat[i-1] +bt[i];
wtavg = wtavg + wt[i];
tatavg = tatavg + tat[i];
}
printf("\n\t PROCESS \tBURST TIME \t WAITING TIME\t TURNAROUND TIME\n");
for(i=0;i<n;i++){
```

```
printf("\n\t P%d \t\t %d \t\t %d \t\t %d", p[i], bt[i], wt[i], tat[i]);
printf("\nAverage Waiting Time -- %f", wtavg/n);
printf("\nAverage Turnaround Time -- %f", tatavg/n);
getch();
}
```

INPUT

Enter the number of processes -- 4
Enter Burst Time for Process 0 -- 6
Enter Burst Time for Process 1 -- 8
Enter Burst Time for Process 2 -- 7
Enter Burst Time for Process 3 – 3

OUTPUT

PROCESS BURST TIME WAITING TIME TURNAROUND TIME
P3 3 0 3
P0 6 3 9
P2 7 9 16
P1 8 16 24
Average Waiting Time -- 7.000000
Average Turnaround Time -- 13.000000

WEEK-2

2. Simulate the following CPU scheduling algorithms a) Priority b) Round robin

ROUND ROBIN CPU SCHEDULING ALGORITHM

```
#include<stdio.h>
main()
{
int i,j,n,bu[10],wa[10],tat[10],t,ct[10],max;
float awt=0,att=0,temp=0;
clrscr();
printf("Enter the no of processes -- ");
scanf("%d",&n);
for(i=0;i<n;i++)
{
printf("\nEnter Burst Time for process %d -- ", i+1);
scanf("%d",&bu[i]);
ct[i]=bu[i];
}
printf("\nEnter the size of time slice -- ");
scanf("%d",&t);
max=bu[0];
for(i=1;i<n;i++)
if(max<bu[i])
max=bu[i];
for(j=0;j<(max/t)+1;j++)
for(i=0;i<n;i++)
if(bu[i]!=0)
if(bu[i]<=t){
tat[i]=temp+bu[i];
temp=temp+bu[i];
bu[i]=0;
}
else{
bu[i]=bu[i]-t;
temp=temp+t;
}
for(i=0;i<n;i++)
{
wa[i]=tat[i]-ct[i];
att+=tat[i];
awt+=wa[i];
}
printf("\nThe Average Turnaround time is -- %f",att/n);
printf("\nThe Average Waiting time is -- %f ",awt/n);
printf("\n\tPROCESS\t BURST TIME \t WAITING TIME\tTURNAROUND TIME\n");
for(i=0;i<n;i++)
```

```
printf("\t%d \t %d \t\t %d \t\t %d \n",i+1,ct[i],wa[i],tat[i]);

getch();
}
```

INPUT

Enter the no of processes – 3
Enter Burst Time for process 1 – 24
Enter Burst Time for process 2 -- 3
Enter Burst Time for process 3 -- 3
Enter the size of time slice – 3

OUTPUT

The Average Turnaround time is – 15.666667
The Average Waiting time is -- 5.666667
PROCESS BURST TIME WAITING TIME TURNAROUND TIME
1 24 6 30
2 3 4 7
3 3 7 10

PRIORITY CPU SCHEDULING ALGORITHM

```
#include<stdio.h>
main()
{
int p[20],bt[20],pri[20], wt[20],tat[20],i, k, n, temp;
float wtavg, tatavg;
clrscr();
printf("Enter the number of processes --- ");
scanf("%d",&n);
for(i=0;i<n;i++)
{
p[i] = i;
printf("Enter the Burst Time & Priority of Process %d --- ",i);
scanf("%d %d",&bt[i], &pri[i]);
}
for(i=0;i<n;i++)
for(k=i+1;k<n;k++)
if(pri[i] > pri[k])
{
temp=p[i];
p[i]=p[k];
p[k]=temp;
temp=bt[i];
bt[i]=bt[k];
bt[k]=temp;
temp=pri[i];
pri[i]=pri[k];
pri[k]=temp;
```

```
}
wtavg = wt[0] = 0;
tatavg = tat[0] = bt[0];
for(i=1;i<n;i++)
{
wt[i] = wt[i-1] + bt[i-1];
tat[i] = tat[i-1] + bt[i];
wtavg = wtavg + wt[i];
tatavg = tatavg + tat[i];
}
printf("\nPROCESS\t\tPRIORITY\tBURST TIME\tWAITING TIME\tTURNAROUND TIME");
for(i=0;i<n;i++)
printf("\n%d \t\t %d \t\t %d \t\t %d \t\t %d ",p[i],pri[i],bt[i],wt[i],tat[i]);
printf("\nAverage Waiting Time is --- %f",wtavg/n);
printf("\nAverage Turnaround Time is --- %f",tatavg/n);
getch();
}
```

INPUT

Enter the number of processes -- 5
Enter the Burst Time & Priority of Process 0 --- 10 3
Enter the Burst Time & Priority of Process 1 --- 1 1
Enter the Burst Time & Priority of Process 2 --- 2 4
Enter the Burst Time & Priority of Process 3 --- 1 5
Enter the Burst Time & Priority of Process 4 --- 5 2

OUTPUT

PROCESS PRIORITY BURST TIME WAITING TIME TURNAROUND TIME
1 1 1 0 1
4 2 5 1 6
0 3 10 6 16
2 4 2 16 18
3 5 1 18 19
Average Waiting Time is --- 8.200000
Average Turnaround Time is --- 12.000000

WEEK-3

3. Simulate Bankers Algorithm for Deadlock Avoidance.

```
#include<stdio.h>
struct file
{
int all[10];
int max[10];
int need[10];
int flag;
};
void main()
{
struct file f[10];
int fl;
int i, j, k, p, b, n, r, g, cnt=0, id, newr;
int avail[10],seq[10];
clrscr();
printf("Enter number of processes -- ");
scanf("%d",&n);
printf("Enter number of resources -- ");
scanf("%d",&r);
for(i=0;i<n;i++)
{
printf("Enter details for P%d",i);
printf("\nEnter allocation\t -- \t");
for(j=0;j<r;j++)
scanf("%d",&f[i].all[j]);
printf("Enter Max\t\t -- \t");
for(j=0;j<r;j++)
scanf("%d",&f[i].max[j]);
f[i].flag=0;
}
printf("\nEnter Available Resources\t -- \t");
for(i=0;i<r;i++)
scanf("%d",&avail[i]);
printf("\nEnter New Request Details -- ");
printf("\nEnter pid \t -- \t");
scanf("%d",&id);
printf("Enter Request for Resources \t -- \t");
for(i=0;i<r;i++)
{
scanf("%d",&newr);
f[id].all[i] += newr;
avail[i]=avail[i] - newr;
```

```
}
for(i=0;i<n;i++)
{
for(j=0;j<r;j++)
{
cnt=0;
fl=0;

f[i].need[j]=f[i].max[j]-f[i].all[j];
if(f[i].need[j]<0)
}
}
while(cnt!=n)
{
g=0;
for(j=0;j<n;j++)
{
if(f[j].flag==0)
{
b=0;
for(p=0;p<r;p++)
{
if(avail[p]>=f[j].need[p])
b=b+1;
else
b=b-1;
}
if(b==r)
{
printf("\nP%d is visited",j);
seq[fl++]=j;
f[j].flag=1;
for(k=0;k<r;k++)
avail[k]=avail[k]+f[j].all[k];
cnt=cnt+1;
printf("(");
for(k=0;k<r;k++)
printf("%3d",avail[k]);
printf(")");
g=1;
}
}
}
if(g==0)
printf("\n REQUEST NOT GRANTED -- DEADLOCK OCCURRED");
printf("\n SYSTEM IS IN UNSAFE STATE");
goto y;
```

```
}
}
printf("\nSYSTEM IS IN SAFE STATE");
printf("\nThe Safe Sequence is -- (");
for(i=0;i<fl;i++)
for(i=0;i<fl;i++)
printf("P%d ",seq[i]);
printf("\nProcess\t\tAllocation\t\tMax\t\t\tNeed\n");
for(i=0;i<n;i++)
{
printf("P%d\t",i);
for(j=0;j<r;j++)
printf("%6d",f[i].all[j]);
for(j=0;j<r;j++)
printf("%6d",f[i].max[j]);
for(j=0;j<r;j++)
printf("%6d",f[i].need[j]);
printf("\n");
}
getch();
}
```

INPUT

```
Enter number of processes – 5
Enter number ofresources
Enter details for P0-- 3
Enter allocation -- 0 1 0
Enter Max -- 7 5 3
Enter details for P1
Enter allocation -- 2 0 0
Enter Max -- 3 2 2
Enter details for P2
Enter allocation -- 3 0 2
Enter Max -- 9 0 2
Enter details for P3
Enter allocation -- 2 1 1
Enter Max -- 2 2 2
Enter details for P4
Enter allocation -- 0 0 2
Enter Max -- 4 3 3
Enter Available Resources -- 3 3 2
Enter New Request Details  --
Enter pid -- 1
Enter Request for Resources -- 1 0 2
```

OUTPUT

```
P1 is visited( 5 3 2)
P3 is visited( 7 4 3)
P4 is visited( 7 4 5)
P0 is visited( 7 5 5)
P2 is visited( 10 5 7)
SYSTEM IS IN SAFE STATE
The Safe Sequence is -- (P1 P3 P4 P0 P2 )
Process Allocation Max Need
P0 0 1 0 7 5 3 7 4 3
P1 3 0 2 3 2 2 0 2 0
P2 3 0 2 9 0 2 6 0 0
P3 2 1 1 2 2 2 0 1 1
P4 0 0 2 4 3 3 4 3 1
```

WEEK-4

4. Implementation of the following Memory Allocation Methods for fixed partition
a) First Fit b) Worst Fit c) Best Fit

FIRST-FIT

```
#include<stdio.h>
#include<conio.h>
#define max 25
void main()
{
int frag[max],b[max],f[max],i,j,nb,nf,temp,highest=0;
static int bf[max],ff[max];
clrscr();
printf("\n\tMemory Management Scheme - Worst Fit");
printf("\nEnter the number of blocks:");
scanf("%d",&nb);
printf("Enter the number of files:");
scanf("%d",&nf);
printf("\nEnter the size of the blocks:-\n");
for(i=1;i<=nb;i++)
{
printf("Block %d:",i);
scanf("%d",&b[i]);
}
printf("Enter the size of the files :-\n");
for(i=1;i<=nf;i++)
{
printf("File %d:",i);
scanf("%d",&f[i]);
}
for(i=1;i<=nf;i++)
{
for(j=1;j<=nb;j++)
{
if(bf[j]!=1) //if bf[j] is not allocated
{
temp=b[j]-f[i];
if(temp>=0)
if(highest<temp)
{
frag[i]=highest;
bf[ff[i]]=1;
highest=0;
}
ff[i]=j;
highest=temp;
```

```
}
printf("\nFile_no:\tFile_size :\tBlock_no:\tBlock_size:\tFragement");
for(i=1;i<=nf;i++)
printf("\n%d\t\t%d\t\t%d\t\t%d\t\t%d",i,f[i],ff[i],b[ff[i]],frag[i]);
getch();
}
```

INPUT

Enter the number of blocks: 3
Enter the number of files: 2
Enter the size of the blocks:-
Block 1: 5
Block 2: 2
Block 3: 7
Enter the size of the files:-
File 1: 1
File 2: 4

OUTPUT

File No File Size Block No Block Size Fragment
1 1 3 7 6
2 4 1 5 1

WORST-FIT

```
#inclUde<stdio.h>
#incude<conio.h>
#define max 25
void main()
{
 int frag[max],b[max],f[max],i,j,nb,nf,temp;
 static int bf[max],ff[max];
 clrscr();
 printf("\n\tMemory Management Scheme - First Fit");
 printf("\nEnter the number of blocks:");
 scanf("%d",&nb);
 printf("\nEnter the size of the blocks:-\n");
for(i=1;i<=nb;i++)
{
printf("Block %d:",i);
scanf("%d",&b[i]);
}
printf("Enter the size of the files :-\n");
for(i=1;i<=nf;i++)
{
printf("File %d:",i);
```

```
scanf("%d",&f[i]);
}
for(i=1;i<=nf;i++)
{
for(j=1;j<=nb;j++)
{
if(bf[j]!=1)
{
temp=b[j]-f[i];
if(temp>=0)
{
ff[i]=j;
break;
}
}
}
frag[i]=temp;
bf[ff[i]]=1;
}
printf("\nFile_no:\tFile_size :\tBlock_no:\tBlock_size:\tFragement");
for(i=1;i<=nf;i++)
printf("\n%d\t\t%d\t\t%d\t\t%d\t\t%d",i,f[i],ff[i],b[ff[i]],frag[i]);
getch();
}
```

INPUT

Enter the number of blocks: 3
Enter the number of files: 2
Enter the size of the blocks:-
Block 1: 5
Block 2: 2
Block 3: 7
Enter the size of the files:-
File 1: 1
File 2: 4

OUTPUT

File No File Size Block No Block Size Fragment
1 1 1 5 4
2 4 3 7 3

BEST-FIT

```
#include<stdio.h>
#include<conio.h>
#define max 25
void main()
{
```

```
int frag[max],b[max],f[max],i,j,nb,nf,temp,lowest=10000;
static int bf[max],ff[max];
clrscr();
printf("\nEnter the number of blocks:");
scanf("%d",&nb);
printf("Enter the number of files:");
scanf("%d",&nf);
printf("\nEnter the size of the blocks:-\n");
for(i=1;i<=nb;i++)
printf("Block %d:",i);scanf("%d",&b[i]);
printf("Enter the size of the files :-\n");
for(i=1;i<=nf;i++)
{
printf("File %d:",i);
scanf("%d",&f[i]);
}
for(i=1;i<=nf;i++)
{
for(j=1;j<=nb;j++)
{
if(bf[j]!=1)
{
temp=b[j]-f[i];
if(temp>=0)
if(lowest>temp)
{
ff[i]=j;
lowest=temp;
}

}
}
frag[i]=lowest;
bf[ff[i]]=1;
lowest=10000;
}
printf("\nFile No\tFile Size \tBlock No\tBlock Size\tFragment");
for(i=1;i<=nf && ff[i]!=0;i++)
printf("\n%d\t\t%d\t\t%d\t\t%d\t\t%d",i,f[i],ff[i],b[ff[i]],frag[i]);
getch();
}
```

INPUT
Enter the number of blocks: 3
Enter the number of files: 2
Enter the size of the blocks:-
Block 1: 5
Block 2: 2
Block 3: 7
Enter the size of the files:-
File 1: 1
File 2: 4

OUTPUT
File No File Size Block No Block Size Fragment
1 1 2 2 1
2 4 1 5 1

WEEK-5

Implementation of Paging Technique of Memory Management

BEST-FIT

```
#include<stdio.h>
#include<conio.h>
#define max 25
void main()
{
int frag[max],b[max],f[max],i,j,nb,nf,temp,lowest=10000;
static int bf[max],ff[max];
clrscr();
printf("\nEnter the number of blocks:");
scanf("%d",&nb);
printf("Enter the number of files:");
scanf("%d",&nf);
printf("\nEnter the size of the blocks:-\n");
for(i=1;i<=nb;i++)
printf("Block %d:",i);scanf("%d",&b[i]);
printf("Enter the size of the files :-\n");
for(i=1;i<=nf;i++)
{
printf("File %d:",i);
scanf("%d",&f[i]);
}
for(i=1;i<=nf;i++)
{
for(j=1;j<=nb;j++)
{
if(bf[j]!=1)
{
temp=b[j]-f[i];
if(temp>=0)
if(lowest>temp)
{
ff[i]=j;
lowest=temp;
}
}
}
frag[i]=lowest;
bf[ff[i]]=1;
lowest=10000;}
```

```
printf("\nFile No\tFile Size \tBlock No\tBlock Size\tFragment");
for(i=1;i<=nf && ff[i]!=0;i++)
printf("\n%d\t\t%d\t\t%d\t\t%d\t\t%d",i,f[i],ff[i],b[ff[i]],frag[i]);
getch();
}
```

INPUT

Enter the number of blocks: 3
Enter the number of files: 2

Enter the size of the blocks:-
Block 1: 5
Block 2: 2
Block 3: 7

Enter the size of the files:-
File 1: 1
File 2: 4

OUTPUT

File No File Size Block No Block Size Fragment 1 1 2 2 1 2 4 1 5 1

FIRST-FIT

```
#include<stdio.h>
#include<conio.h>
#define max 25

void main()
{
int frag[max],b[max],f[max],i,j,nb,nf,temp;
static int bf[max],ff[max];
clrscr();

printf("\n\tMemory Management Scheme - First Fit");
printf("\nEnter the number of blocks:");
scanf("%d",&nb);
printf("Enter the number of files:");
scanf("%d",&nf);
printf("\nEnter the size of the blocks:-\n");
for(i=1;i<=nb;i++)
{
printf("Block %d:",i);
scanf("%d",&b[i]);
}
printf("Enter the size of the files :-\n");
```

```
for(i=1;i<=nf;i++)
{
printf("File %d:",i);
scanf("%d",&f[i]);
}
for(i=1;i<=nf;i++)
{
for(j=1;j<=nb;j++)
{
if(bf[j]!=1)
{
temp=b[j]-f[i];
if(temp>=0)
{
ff[i]=j;
break;
}
}
}
frag[i]=temp;
bf[ff[i]]=1;
}
printf("\nFile_no:\tFile_size :\tBlock_no:\tBlock_size:\tFragement");
for(i=1;i<=nf;i++)
printf("\n%d\t\t%d\t\t%d\t\t%d\t\t%d",i,f[i],ff[i],b[ff[i]],frag[i]);
getch();
}
```

INPUT

Enter the number of blocks: 3
Enter the number of files: 2

Enter the size of the blocks:-
Block 1: 5
Block 2: 2
Block 3: 7

Enter the size of the files:-
File 1: 1
File 2: 4

OUTPUT

File No File Size Block No Block Size Fragment 1 1 1 5 4 2 4 3 7 3

WORST-FIT

```
#include<stdio.h>
#include<conio.h>
#define max 25

void main()
{
int frag[max],b[max],f[max],i,j,nb,nf,temp,highest=0;
static int bf[max],ff[max];
clrscr();

printf("\n\tMemory Management Scheme - Worst Fit");
printf("\nEnter the number of blocks:");
scanf("%d",&nb);
printf("Enter the number of files:");
scanf("%d",&nf);
printf("\nEnter the size of the blocks:-\n");
for(i=1;i<=nb;i++)
{
printf("Block %d:",i);
scanf("%d",&b[i]);
}
printf("Enter the size of the files :-\n");
for(i=1;i<=nf;i++)
{
printf("File %d:",i);
scanf("%d",&f[i]);
}
for(i=1;i<=nf;i++)
{
for(j=1;j<=nb;j++)
{
if(bf[j]!=1) //if bf[j] is not allocated
{
temp=b[j]-f[i];
if(temp>=0)
if(highest<temp)
{
ff[i]=j;
highest=temp;
}
}
}
frag[i]=highest;
bf[ff[i]]=1;
highest=0;
```

```
}
printf("\nFile_no:\tFile_size :\tBlock_no:\tBlock_size:\tFragement");
for(i=1;i<=nf;i++)
printf("\n%d\t\t%d\t\t%d\t\t%d\t\t%d",i,f[i],ff[i],b[ff[i]],frag[i]);
getch();
}
```

INPUT

Enter the number of blocks: 3
Enter the number of files: 2

Enter the size of the blocks:-
Block 1: 5
Block 2: 2
Block 3: 7

Enter the size of the files:-
File 1: 1
File 2: 4

OUTPUT

File No File Size Block No Block Size Fragment 1 1 3 7 6 2 4 1 5 1

WEEK-6

Implementation of the following Page Replacement Algorithms a) FIFO b) LRU c)optimal

FIFO PAGE REPLACEMENT ALGORITHM

```
#include<stdio.h>
#include<conio.h>
main()
{
int i, j, k, f, pf=0, count=0, rs[25], m[10], n;
clrscr();
printf("\n Enter the length of reference string -- ");
scanf("%d",&n);
printf("\n Enter the reference string -- ");
for(i=0;i<n;i++)
scanf("%d",&rs[i]);
printf("\n Enter no. of frames -- ");
scanf("%d",&f);
for(i=0;i<f;i++)
m[i]=-1;
printf("\n The Page Replacement Process is -- \n");
for(i=0;i<n;i++)
{
for(k=0;k<f;k++)
{
if(m[k]==rs[i])
break;
}
if(k==f)
{
m[count++]=rs[i];
pf++;
}
for(j=0;j<f;j++)
printf("\t%d",m[j]);
if(k==f)
printf("\tPF No. %d",pf);
printf("\n");
if(count==f)
count=0;
}
printf("\n The number of Page Faults using FIFO are %d",pf);
getch();
}
```

INPUT
Enter the length of reference string – 20
Enter the reference string -- 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1 Enter no. of frames -- 3

OUTPUT
The Page Replacement Process is –
7 -1 -1 PF No. 1
7 0 -1 PF No. 2
7 0 1 PF No. 3
2 0 1 PF No. 4
2 0 1
2 3 1 PF No. 5
2 3 0 PF No. 6
4 3 0 PF No. 7
4 2 0 PF No. 8
4 2 3 PF No. 9
0 2 3 PF No. 10
0 2 3
0 2 3
0 1 3 PF No. 11
0 1 2 PF No. 12
0 1 2
0 1 2
7 1 2 PF No. 13
7 0 2 PF No. 14
7 0 1 PF No. 15

The number of Page Faults using FIFO are 15

LRU PAGE REPLACEMENT ALGORITHM

```
#include<stdio.h>
#include<conio.h>
main()
{
int i, j , k, min, rs[25], m[10], count[10], flag[25], n, f, pf=0, next=1;  clrscr();
printf("Enter the length of reference string -- ");
scanf("%d",&n);
printf("Enter the reference string -- ");
for(i=0;i<n;i++)
{
scanf("%d",&rs[i]);
flag[i]=0;
}
printf("Enter the number of frames -- ");
scanf("%d",&f);
for(i=0;i<f;i++)
```

```
{
count[i]=0;
m[i]=-1;
}
printf("\nThe Page Replacement process is -- \n");
for(i=0;i<n;i++)
{
for(j=0;j<f;j++)
{
if(m[j]==rs[i])
{
flag[i]=1

}
}
if(flag[i]==0) {
if(i<f)
{
}
else
{
}
pf++;
}
for(j=0;j<f;j++)
count[j]=next;
next++;
m[i]=rs[i];
count[i]=next;
next++;
min=0;
for(j=1;j<f;j++)
if(count[min] > count[j])
min=j;
m[min]=rs[i];
count[min]=next;
next++;

printf("%d\t",m[j]);
if(flag[i]==0)
printf("PF No. -- %d" , pf);
printf("\n");
}
printf("\nThe number of page faults using LRU are %d",pf);  getch();
}
```

INPUT

Enter the length of reference string -- 20
Enter the reference string -- 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1 Enter the number of frames -- 3

OUTPUT

The Page Replacement process is --

```
7   -1   -1       PF No. -- 1
7   0    -1       PF No. -- 2
7   0    1        PF No. -- 3
2   0    1        PF No. -- 4
2   0    1
2   0    3        PF No. -- 5
2   0  3
4   0    3        PF No. – 6
4   0    2        PF No. – 7
4   3    2        PF No. – 8
0   3  2          PF No. – 9
0   3    2
0   3    2
1   3    2        PF No. – 10
1   3    2
1   0    2        PF No. – 11
1   0    2
1   0    7        PF No. – 12
1   0    7
1   0    7
```

The number of page faults using LRU are 12

LFU PAGE REPLACEMENT ALGORITHM

```
#include<stdio.h>
#include<conio.h>
main()
{
int rs[50], i, j, k, m, f, cntr[20], a[20], min, pf=0;
clrscr();
printf("\nEnter number of page references -- ");
scanf("%d",&m);
printf("\nEnter the reference string -- ");
for(i=0;i<m;i++)
scanf("%d",&rs[i]);
printf("\nEnter the available no. of frames -- ");
scanf("%d",&f);
for(i=0;i<f;i++)
{
cntr[i]=0;
```

```
a[i]=-1;
}
Printf("\nThe Page Replacement Process is – \n");
for(i=0;i<m;i++)
{
for(j=0;j<f;j++)
if(rs[i]==a[j])
{
cntr[j]++;
break;
}
if(j==f)
{
min = 0;
for(k=1;k<f;k++)
if(cntr[k]<cntr[min])
min=k;
a[min]=rs[i];
cntr[min]=1;
pf++;
}
printf("\n");
for(j=0;j<f;j++)
printf("\t%d",a[j]);
if(j==f)
printf("\tPF No. %d",pf);
}
printf("\n\n Total number of page faults -- %d",pf);
getch();
}
```

INPUT

Enter number of page references -- 10
Enter the reference string -- 1 2 3 4 5 2 5 2 5 1 4 3 Enter the available no. of frames--- 3

OUTPUT

The Page Replacement Process is –
1 -1 -1 PF No. 1 1 2 -1 PF No. 2 1 2 3 PF No. 3 4 2 3 PF No. 4 5 2 3 PF No. 5 5 2 3
5 2 3
5 2 1 PF No. 6 5 2 4 PF No. 7 5 2 3 PF No. 8
Total number of page faults -- 8

WEEK-7

Simulate frame allocation Methods.
a) minimum number of frames b) equal allocation c) proportional allocation
1) Minimum number of frames
PROGRAM

```
#include <stdio.h>
#include <stdlib.h>

#define MAX_PROCESSES 100
typedef struct {
    int id;
    int memory;
} Process;

void allocateFrames(int totalMemorySize, int numberOfProcesses, Process processes[], int frameSize) {
    int totalFrames = totalMemorySize / frameSize;
    int allocatedFrames = 0;
    int allocationTable[totalFrames];

    for (int i = 0; i < totalFrames; i++) {
        allocationTable[i] = -1; // -1 indicates free frame
    }

        for (int i = 0; i < numberOfProcesses - 1; i++) {
        for (int j = 0; j < numberOfProcesses - i - 1; j++) {
            if (processes[j].memory > processes[j + 1].memory) {
                Process temp = processes[j];
                processes[j] = processes[j + 1];
                processes[j + 1] = temp;
            }
        }
    }

        for (int i = 0; i < numberOfProcesses; i++) {
        int requiredFrames = processes[i].memory / frameSize;
        if (allocatedFrames + requiredFrames > totalFrames) {
            break;
        }
        for (int j = 0; j < requiredFrames; j++) {
            allocationTable[allocatedFrames + j] = processes[i].id;
        }
        allocatedFrames += requiredFrames;
    }

    printf("Memory Allocation Table:\n");
    for (int i = 0; i < totalFrames; i++) {
        if (allocationTable[i] == -1) {
```

```
            printf("Frame %d: Free\n", i);
        } else {
            printf("Frame %d: Process %d\n", i, allocationTable[i]);
        }
    }
}

int main() {
    int totalMemorySize = 1024; // in KB
    int numberOfProcesses = 3;
    Process processes[MAX_PROCESSES] = {
        {1, 400}, // Process 1: 400 KB
        {2, 200}, // Process 2: 200 KB
        {3, 300}  // Process 3: 300 KB
    };
    int frameSize = 100; // in KB
    allocateFrames(totalMemorySize, numberOfProcesses, processes, frameSize);
    return 0;
}
```

INPUT:

Total Memory Size = 1024 KB

Number of Processes = 3

Memory Requirements:

Process 1: 400 KB

Process 2: 200 KB

Process 3: 300 KB

Frame Size = 100 KB

OUTPUT:

Memory Allocation Table: Frame 0-3: Process 1

Frame 4-5: Process 2

Frame 6-8: Process 3

2) Equal Allocation

PROGRAM

```
#include <stdio.h>
#define MAX_PROCESSES 100
typedef struct {
    int id;
    int memory;
} Process;

void equalAllocation(int totalMemorySize, int numberOfProcesses, Process processes[]) {
    int memoryPerProcess = totalMemorySize / numberOfProcesses;

    printf("Memory Allocation Table:\n");
    for (int i = 0; i < numberOfProcesses; i++) {
        printf("Process %d: Memory allocated = %d\n", processes[i].id, memoryPerProcess);
    }
}

int main() {
    int totalMemorySize = 1024; // Total memory size in KB
    int numberOfProcesses = 3;
    Process processes[MAX_PROCESSES] = {
        {1, 0}, // Process IDs start from 1
        {2, 0},
        {3, 0}
};

    for (int i = 0; i < numberOfProcesses; i++) {
        printf("Enter memory requirement for Process %d (in KB): ", i + 1);
        scanf("%d", &processes[i].memory);
    }
    equalAllocation(totalMemorySize, numberOfProcesses, processes);
    return 0;
}
```

INPUT:

Total memory size: 1024 KB
Number of processes: 3
Memory requirement for Process 1 (in KB): 300
Memory requirement for Process 2 (in KB): 400
Memory requirement for Process 3 (in KB): 200

OUTPUT:

Memory Allocation Table:
Process 1: Memory allocated = 341

Process 2: Memory allocated = 341
Process 3: Memory allocated = 341

3) Proportional Allocation
PROGRAM

```
#include <stdio.h>
#define MAX_PROCESSES 100

typedef struct {
    int id;
    int memory;
} Process;

void proportionalAllocation(int totalMemorySize, int numberOfProcesses, Process processes[]) {
    int totalMemoryRequirements = 0;

    for (int i = 0; i < numberOfProcesses; i++) {
        totalMemoryRequirements += processes[i].memory;
    }
    printf("Memory Allocation Table:\n");
    for (int i = 0; i < numberOfProcesses; i++) {
        float allocation = (float)(processes[i].memory * totalMemorySize) / totalMemoryRequirements;
        printf("Process %d: Memory allocated = %.2f KB\n", processes[i].id, allocation);
    }
}
int main() {
    int totalMemorySize = 1024; // Total memory size in KB
    int numberOfProcesses = 3;
    Process processes[MAX_PROCESSES] = {
        {1, 0}, // Process IDs start from 1
        {2, 0},
        {3, 0}
    };
    // Input memory requirements for each process
    for (int i = 0; i < numberOfProcesses; i++) {
        printf("Enter memory requirement for Process %d (in KB): ", i + 1);
        scanf("%d", &processes[i].memory);
    }
    Proportional Allocation(totalMemorySize, numberOfProcesses, processes);
    return 0;
}
```

INPUT:
Total memory size: 1024 KB
Number of processes: 3
Memory requirement for Process 1 (in KB): 300
Memory requirement for Process 2 (in KB): 400
Memory requirement for Process 3 (in KB): 200

OUTPUT:
Memory Allocation Table:
Process 1: Memory allocated = 312.89 KB
Process 2: Memory allocated = 417.19 KB
Process 3: Memory allocated = 208.91 KB

WEEK-8

Write a C program to simulate disk scheduling algorithms
a) FCFS b) SCAN c) C-SCAN
FCFS DISK SCHEDULING ALGORITHM
PROGRAM

```
#include<stdio.h>
main()
{
int t[20], n, I, j, tohm[20], tot=0;
float avhm;
clrscr();
printf("enter the no.of tracks");
scanf("%d",&n);
printf("enter the tracks to be traversed");
for(i=2;i<n+2;i++)
scanf("%d",&t*i+);
for(i=1;i<n+1;i++)
{
tohm[i]=t[i+1]-t[i];
if(tohm[i]<0)
tohm[i]=tohm[i]*(-1);
}
for(i=1;i<n+1;i++)
tot+=tohm[i];
avhm=(float)tot/n;
printf("Tracks traversed\tDifference between tracks\n");
for(i=1;i<n+1;i++)
printf("%d\t\t\t%d\n",t*i+,tohm*i+);
printf("\nAverage header movements:%f",avhm);
getch(); }
```

INPUT:
Enter no. of tracks: 9
Enter track position: 55 58 60 70 18 90 150 160 184

OUTPUT:

Tracks traversed	Difference between tracks
55	45
58	3
60	2
70	10
18	52
90	72
150	60
160	10
184	24

Average header movements: 30. 888889

2.SCAN DISK SCHEDULING ALGORITHM
PROGRAM

```
#include<stdio.h>
main()
{
int t[20], d[20], h, i, j, n, temp, k, atr[20], tot, p, sum=0;
clrscr();
printf("enter the no of tracks to be traveresed");
scanf("%d'",&n);
printf("enter the position of head");
scanf("%d",&h);
t[0]=0;t[1]=h;
printf("enter the tracks");
for(i=2;i<n+2;i++)
        scanf("%d",&t[i]);
for(i=0;i<n+2;i++)
{
        for(j=0;j<(n+2)-i-1;j++)
                { if(t[j]>t[j+1])
                        { temp=t[j];
                        t[j]=t[j+1];
                        t[j+1]=temp;
} } }
for(i=0;i<n+2;i++)
        if(t[i]==h)
                j=i;k=i;

p=0;
while(t[j]!=0)
{
        atr[p]=t[j];
        j--;
        p++;
}
atr[p]=t[j];
for(p=k+1;p<n+2;p++,k++)
        atr[p]=t[k+1];
for(j=0;j<n+1;j++)
{
        if(atr[j]>atr[j+1])
                d[j]=atr[j]-atr[j+1];
        else
                d[j]=atr[j+1]-atr[j];
sum+=d[j];
}
printf("\nAverage header movements:%f",(float)sum/n);
```

```
getch();
}
```

INPUT

Enter no. of tracks:9
Enter track position: 55 58 60 70 18 90 150 160 184

OUTPUT

Tracks traversed	Difference between tracks
150	50
160	10
184	24
90	94
70	20
60	10
58	2
55	3
18	37

Average header movements: 27.77

3 .C-SCAN DISK SCHEDULING ALGORITHM

```
#include<stdio.h>
main()
{
        int t[20], d[20], h, i, j, n, temp, k, atr[20], tot, p, sum=0;
clrscr();
printf("enter the no of tracks to be traveresed");
scanf("%d'",&n);
printf("enter the position of head");
scanf("%d",&h);
t[0]=0;t[1]=h;
printf("enter total tracks");
scanf("%d",&tot);
t[2]=tot-1;
printf("enter the tracks");
for(i=3;i<=n+2;i++)
        scanf("%d",&t[i]);
for(i=0;i<=n+2;i++)
        for(j=0;j<=(n+2)-i-1;j++)
                if(t[j]>t[j+1])
                {
                        temp=t[j];
                        t[j]=t[j+1];
                        t[j+1]=temp;
                }
for(i=0;i<=n+2;i++)
        if(t[i]==h)
```

```
                j=i;break;
p=0;
while(t[j]!=tot-1)
{
        atr[p]=t[j];
        j++;
        p++;
        }
        atr[p]=t[j];
        p++;
        i=0;
        while(p!=(n+3) && t[i]!=t[h])
        {
                atr[p]=t[i];
                i++;
                p++;
        }
        for(j=0;j<n+2;j++)
        {
                if(atr[j]>atr[j+1])
                        d[j]=atr[j]-atr[j+1];
                else
                        d[j]=atr[j+1]-atr[j];
                sum+=d[j];
}
        printf("total header movements%d",sum);
        printf("avg is %f",(float)sum/n);
        getch();
}
```

INPUT

Enter the track position : 55 58 60 70 18 90 150 160 184
Enter starting position : 100

OUTPUT

Tracks traversed	Difference between tracks
150	50
160	10
184	24
18	240
55	37
58	3
60	2
70	10
90	20

Average seek time : 35.7777779

WEEK-9

9. Simulate the following Disk scheduling algorithms.
a) C-SCAN b) LOOK c) C-LOOK

a) C-SCAN Program :

```
#include <stdlib.h>
void cscan(int arr[], int head, int size) {
    const int disk_range = 200;
    int total_movement = 0;
    int start = 0;
    int end = disk_range - 1;
    printf("Disk Head Movement: %d", head);
    for (int i = 0; i < size; i++) {
        if (arr[i] >= head) {
            total_movement += arr[i] - head;
            printf(" -> %d", arr[i]);
            head = arr[i];
        }
    }
    total_movement += end - head;
    printf(" -> %d", end);
    total_movement += end - start;
    printf(" -> %d", start);
    for (int i = 0; i < size; i++) {
        if (arr[i] < head) {
            total_movement += head - arr[i];
            printf(" -> %d", arr[i]);
            head = arr[i];
        }
    }
    printf("\nTotal Head Movement: %d\n", total_movement);
}
int main() {
    int size;
    printf("Enter the size of the queue: ");
    scanf("%d", &size);

    int arr[size];
    printf("Enter the requests: ");
    for (int i = 0; i < size; i++) {
        scanf("%d", &arr[i]);}
```

```
    int head;
    printf("Enter the initial position of the head: ");
    scanf("%d", &head);
    cscan(arr, head, size);
    return 0;
}
```

INPUT :

Enter the size of the queue: 8
Enter the requests: 98 183 37 122 14 124 65 67
Enter the initial position of the head: 53

OUTPUT :

Disk Head Movement: 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 199 -> 0 -> 14 -> Total Head Movement: 353

b) LOOK Program :

```
#include <stdio.h>
#include <stdlib.h>
void look(int arr[], int head, int size) {
    int total_movement = 0;
    int start = 0;
    int end = 199; // Assuming the disk range is 0-199
    printf("Disk Head Movement: %d", head);
    for (int i = 0; i < size; i++) {
        if (arr[i] >= head) {
            total_movement += arr[i] - head;
            printf(" -> %d", arr[i]);
            head = arr[i];
        }
    }
    for (int i = size - 1; i >= 0; i--) {
        if (arr[i] < head) {
            total_movement += head - arr[i];
            printf(" -> %d", arr[i]);
            head = arr[i];
        }
    }
```

```
    printf("\nTotal Head Movement: %d\n", total_movement);
}
int main() {
    int size;
    printf("Enter the size of the queue: ");
    scanf("%d", &size);
    int arr[size];
    printf("Enter the requests: ");
    for (int i = 0; i < size; i++) {
        scanf("%d", &arr[i]);
    }
    int head;
    printf("Enter the initial position of the head: ");
    scanf("%d", &head);
    look(arr, head, size);
    return 0;
}
```

INPUT :

Enter the size of the queue: 8
Enter the requests: 98 183 37 122 14 124 65 67
Enter the initial position of the head: 53

OUTPUT :

Disk Head Movement: 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 14 -> Total Head Movement: 236

c) C-LOOK Program :

```
#include <stdio.h>
#include <stdlib.h>
void clook(int arr[], int head, int size) {
    int total_movement = 0;
    int start = 0;
    int end = 199; // Assuming the disk range is 0-199

    printf("Disk Head Movement: %d", head);
    for (int i = 0; i < size; i++) {
        if (arr[i] >= head) {
            total_movement += arr[i] - head;
            printf(" -> %d", arr[i]);
            head = arr[i];
        }
```

```
    }
    if (head != end) {
        total_movement += end - head;
        printf(" -> %d", end);
        head = end;
    }
    for (int i = 0; i < size; i++) {
        if (arr[i] < head) {
            total_movement += head - arr[i];
            printf(" -> %d", arr[i]);
            head = arr[i];
        }
    }
    printf("\nTotal Head Movement: %d\n", total_movement);
}
int main() {
    int size;
    printf("Enter the size of the queue: ");
    scanf("%d", &size);
    int arr[size];
    printf("Enter the requests: ");
    for (int i = 0; i < size; i++) {
        scanf("%d", &arr[i]);
    }
    int head;
    printf("Enter the initial position of the head: ");
    scanf("%d", &head);
    clook(arr, head, size);
    return 0;
}
```

INPUT :

Enter the size of the queue: 8
Enter the requests: 98 183 37 122 14 124 65 67
Enter the initial position of the head: 53

OUTPUT :

Disk Head Movement: 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 199 -> 0 -> 14 -> Total Head Movement: 345

WEEK-10

10. Simulate all file allocation strategies.
a) Sequential b) Indexed c) Linked.

a) SEQUENTIAL FILEALLOCATION :

```
#include<stdio.h>
#include<conio.h>

struct fileTable
{
        char name[20];
        int sb, nob;
}ft[30];
void main()
{
        int i, j, n;
        char s[20];
        clrscr();
        printf("Enter no of files :");
        scanf("%d",&n);
        for(i=0;i<n;i++)
        {
                printf("\nEnter file name %d :",i+1);
                scanf("%s",ft[i].name);
                printf("Enter starting block of file %d :",i+1);
                scanf("%d",&ft[i].sb);
                printf("Enter no of blocks in file %d :",i+1);
                scanf("%d",&ft[i].nob);
        }
        printf("\nEnter the file name to be searched -- ");
        scanf("%s",s);
        for(i=0;i<n;i++)
        {
                if(strcmp(s, ft[i].name)==0)
                break;
        }
        if(i==n)
                printf("\nFile Not Found");
        else
        {
                printf("\nFILE NAME START BLOCK NO OF BLOCKS BLOCKS OCCUPIED\n");
```

```
                printf("\n%s\t\t%d\t\t%d\t",ft[i].name,ft[i].sb,ft[i].nob);
                for(j=0;j<ft[i].nob;j++)
                        printf("%d, ",ft[i].sb+j);
        }
        getch();
}
```

INPUT :

Enter no of files :3
Enter file name 1 :A
Enter starting block of file 1 :85
Enter no of blocks in file 1 :6
Enter file name 2 :B
Enter starting block of file 2 :102
Enter no of blocks in file 2 :4
Enter file name 3 :C
Enter starting block of file 3 :60
Enter no of blocks in file 3 :4
Enter the file name to be searched -- B

OUTPUT :

FILE NAME	START BLOCK	NO OF BLOCKS	BLOCKS OCCUPIED
B	102	4	102, 103, 104, 105

b) LINKED FILE ALLOCATION :

```
#include<stdio.h>
#include<conio.h>

struct fileTable
{
char name[20];
int nob;
struct block *sb;
}ft[30];

struct block
{
        int bno;
        struct block *next;
```

```
};
void main()
{
        int i, j, n;
        char s[20];
        struct block *temp;
        clrscr();
        printf("Enter no of files :");
        scanf("%d",&n);
        for(i=0;i<n;i++)
        {
                printf("\nEnter file name %d :",i+1);
                scanf("%s",ft[i].name);
                printf("Enter no of blocks in file %d :",i+1);
                scanf("%d",&ft[i].nob);
                ft[i].sb=(struct block*)malloc(sizeof(struct block));
                temp = ft[i].sb;
                printf("Enter the blocks of the file :");
                scanf("%d",&temp->bno);
                temp->next=NULL;
                for(j=1;j<ft[i].nob;j++)
                {
                        temp->next = (struct block*)malloc(sizeof(struct block));
                        temp = temp->next;
                        scanf("%d",&temp->bno);
                }
                temp->next = NULL;
        }
        printf("\nEnter the file name to be searched -- ");
        scanf("%s",s);
        for(i=0;i<n;i++)
                if(strcmp(s, ft[i].name)==0)
                        break;
        if(i==n)
                printf("\nFile Not Found");
        else
        {
                printf("\nFILE NAME NO OF BLOCKS BLOCKS OCCUPIED");
                printf("\n %s\t\t%d\t",ft[i].name,ft[i].nob);
                temp=ft[i].sb;
                for(j=0;j<ft[i].nob;j++)
```

```
                {
                        printf("%d □ ",temp->bno);
                        temp = temp->next;
                }
        }
        getch();
}
```

INPUT :

Enter no of files 2
Enter file 1 : A
Enter no of blocks in file 1 4
Enter the blocks of the file 1 : 12 23 9 4
Enter file 2 : G
Enter no of blocks in file 2 5
Enter the blocks of the file 2 88 77 66 55 44
Enter the file to be searched : G

OUTPUT :

FILE NAME	NO OF BLOCKS	BLOCKS OCCUPIED
G	5	88 ->77-> 66->55-> 44

c) INDEXED FILE ALLOCATION

```
#include<stdio.h>
#include<conio.h>

struct fileTable
{
char name[20];
int nob, blocks[30];
}ft[30];

void main()
{
        int i, j, n;
        char s[20];
        clrscr();
        printf("Enter no of files :");
        scanf("%d",&n);
```

```
        for(i=0;i<n;i++)
        {
                printf("\nEnter file name %d :",i+1);
                scanf("%s",ft[i].name);
                printf("Enter no of blocks in file %d :",i+1);
                scanf("%d",&ft[i].nob);
                printf("Enter the blocks of the file :");
                for(j=0;j<ft[i].nob;j++)
                        scanf("%d",&ft[i].blocks[j]);
        }
        printf("\nEnter the file name to be searched -- ");
        scanf("%s",s);
        for(i=0;i<n;i++)
                if(strcmp(s, ft[i].name)==0)
                        break;
        if(i==n)
                printf("\nFile Not Found");
        else
        {
                printf("\nFILE NAME NO OF BLOCKS BLOCKS OCCUPIED");
                printf("\n %s\t\t%d\t",ft[i].name,ft[i].nob);
                for(j=0;j<ft[i].nob;j++)
                        printf("%d, ",ft[i].blocks[j]);
        }
        getch();
}
```

INPUT :

Enter no of files 2
Enter file 1 : A
Enter no of blocks in file 1 4
Enter the blocks of the file 1 : 12 23 9 4
Enter file 2 : G
Enter no of blocks in file 2 5
Enter the blocks of the file 2 88 77 66 55 44
Enter the file to be searched : G

OUTPUT :

FILE NAME	NO OF BLOCKS	BLOCKS OCCUPIED
G	5	88, 77, 66, 55, 44